

IteraTri: Detecting Hallucination of LLM in Knowledge-Intensive QA via Iterative Verification

Yunsong Wang¹, Junwei Zhang², Dongjin Yu¹(✉), and Xiaolin Li²

¹ Hangzhou Dianzi University, Zhejiang, China
yudj@hdu.edu.cn

² Hangzhou Institute of Medicine Chinese Academy of Sciences, Zhejiang, China

Abstract. Large Language Models (LLMs) frequently hallucinate when performing complex, multi-hop reasoning, as they lack a mechanism to ground their reasoning in verifiable facts. Existing solutions either rely on the model’s internal consistency, which cannot correct parametric errors, or perform retrieval to provide contextual knowledge without explicit stepwise verification. To bridge this gap, we propose IteraTri, a novel **I**terative, **T**riplet-Driven Knowledge Acquisition and Verification framework. IteraTri reformulates complex question answering into a dynamic process where an LLM planner iteratively decomposes the question, generating sequences of verifiable (subject, relation, object) triplets as its knowledge needs. A dedicated reflection agent then condenses these triplets into a coherent evidence chain, which is rigorously validated against a trusted knowledge graph. This creates a tight, explainable coupling between neural reasoning and symbolic fact-checking. We evaluate IteraTri on three challenging benchmarks (FactCHD, PathQuestion, MetaQA), where it consistently outperforms the baselines, achieving state-of-the-art factual accuracy. Further more, ablation studies confirm the critical role of both iterative extraction and knowledge reflection. Our work demonstrates that using structured triplets as a medium for planning and verification is a promising path towards more reliable and transparent LLMs for knowledge-intensive QA tasks.

Keywords: Large Language Model · Hallucination detection · Knowledge Graph · Explainable AI · IteraTri.

1 Introduction

The remarkable ability of Large Language Models (LLMs) to comprehend and generate human-like text has propelled them to the forefront of knowledge-intensive question answering (QA) [1]. However, their propensity to produce convincing yet factually incorrect statements, which is a phenomenon known as hallucination [2,3], remains a critical barrier to reliable, real-world deployment, especially in domains requiring complex, multi-step reasoning over precise facts [4,5]. The challenge is exacerbated when models are required to synthesize answers from disparate pieces of knowledge [6], as they may fabricate plausible-sounding connections or entities not grounded in any credible source [7,8].

To combat hallucination, existing research has evolved along two primary paradigms. The first paradigm seeks to **ground LLMs in external knowledge sources** [9]. This includes Retrieval-Augmented Generation (RAG), which augments prompts with retrieved document passages, and Knowledge Graph-based QA (KGQA), which executes precise symbolic queries over structured graphs [10]. While promising, these methods face inherent trade-offs: RAG often yields **imprecise** evidence from unstructured text, complicating direct fact verification, while KGQA systems suffer from **inflexibility**, struggling to generalize to complex questions requiring dynamic, multi-hop reasoning paths. The second paradigm attempts to **elicit self-assessment from the LLM itself**, leveraging its inherent parametric knowledge and reasoning capability [11]. Methods in this vein, such as consistency sampling [12], self-interrogation [13], or multi-agent debate [14], detect hallucinations by analyzing the model’s own output variations or internal consistency. However, these approaches are fundamentally constrained by the model’s **prior beliefs and systematic biases**; they may fail to correct errors that are consistently and confidently reproduced by the model itself, and they offer little insight into which specific fact is at fault when an error is detected.

In essence, a significant gap persists between **flexible but ungrounded self-verification** and **precise but rigid external knowledge access**. Current hybrid approaches that loosely combine these elements lack a tight, verifiable coupling between the iterative reasoning process and atomic factual evidence. This results in limited **explainability** and persistent **hallucination** in complex reasoning scenarios.

To bridge this gap, we introduce **IteraTri**, a novel framework for Iterative, Triplet-Driven Knowledge Acquisition and Verification. IteraTri reformulates complex reasoning as an iterative process that involves the generation and verification of structured knowledge needs through a three-stage pipeline. First, an LLM planner decomposes the question via multi-round deliberation, each step producing a natural language query and—critically—the expected answer in the form of structured (**subject, relation, object**) triplets. Second, a distinct LLM reflection agent condenses this extracted knowledge, selecting only the triplets that form a minimal, coherent evidence chain. Third, each triplet in this chain is directly verified against a trusted Knowledge Graph, and a factuality judgment is made based on the verification rate. This methodology ensures that the final verdict is not only derived from a transparent reasoning trace but also built upon a verified factual foundation.

Our primary contributions are threefold:

- We propose **IteraTri**, a novel framework that uses structured triplets as the central medium for iterative reasoning planning and explicit fact verification, effectively bridging flexible neural reasoning with precise symbolic grounding for hallucination detection.
- We introduce a dedicated **knowledge reflection and condensation** step, which significantly enhances the relevance and precision of the evidence chain before verification, improving both efficiency and accuracy.

- We conduct comprehensive experiments on multiple hallucination detection benchmarks. Results demonstrate that IteraTri achieves superior detection accuracy compared to state-of-the-art baselines, while providing fully explainable judgment paths grounded in verified facts.

2 Relate Work

Our work sits at the intersection of hallucination detection in large language models, knowledge-augmented reasoning. We review the most relevant literature in these areas and highlight the distinctive position of our proposed IteraTri framework.

2.1 Hallucination Detection and Mitigation in LLMs

The problem of hallucination has spurred numerous detection and mitigation strategies. These can be broadly categorized by their core mechanism. **Internal Consistency** methods, such as SelfCheckGPT [12] and its variants, operate on the principle that a fact is less trustworthy if the LLM gives inconsistent answers across multiple samplings or prompt perturbations. Similarly, **Self-Interrogation** methods like InterrogateLM [13] and related dialogue-based probing techniques ask the model to explain or justify its own claims, detecting hallucination through the quality or consistency of the generated rationale. More recently, **Multi-Agent Debate** frameworks have been proposed, where multiple LLM agents reason collaboratively or adversarially to arrive at a more faithful conclusion, as seen in the TRUTH-TRIANGULATOR baseline [14].

Critical Analysis: While effective in many scenarios, introspective methods are fundamentally constrained by the model’s parametric knowledge. They operate entirely within the model’s internal representations, offering no mechanism to consult external, authoritative sources. Consequently, they may confidently reproduce systematic errors present in the training data, and when a hallucination is detected, they cannot pinpoint which specific fact is at fault. This lack of external grounding and fine-grained explainability limits their reliability for high-stakes applications. By contrast, our work shifts the paradigm from internal consistency checking to external factual verification.

2.2 Knowledge-Augmented Language Models

To ground LLMs in external knowledge, retrieval-augmented generation (RAG) has become the predominant paradigm. It can be categorized according to the form of the knowledge retrieved.[15] The classic and most common approach is **Textual RAG** [16], which enhances prompts with relevant passages retrieved from an unstructured or semi-structured corpus (e.g., Wikipedia). For more complex reasoning, **Iterative RAG** methods interleave retrieval and generation in multiple steps.

A significant advancement is the integration of structured knowledge, leading to **Knowledge Graph-enhanced RAG**. This line of work retrieves information from knowledge graphs, typically by sampling or linearizing relevant subgraphs or by using KG embeddings to find semantically related facts. The retrieved structured data is then serialized into text and fed into the LLM as context. This method aims to combine the precision of structured knowledge with the flexibility of LLMs. [17]

Critical Analysis: KG-enhanced RAG advances beyond pure introspection by incorporating structured knowledge. However, its core mechanism remains one of contextual supplementation: retrieved triples are linearized and fed to the LLM as additional context, with reasoning performed in a single, opaque forward pass. Although this can improve the accuracy of facts, it lacks a built-in mechanism to verify which specific retrieved fact supports which step of reasoning. The final answer, therefore, lacks a verifiable, stepwise link to atomic KG facts. In comparison, IteraTri addresses this by reframing retrieval as a targeted verification of each discrete reasoning step, ensuring full traceability.

Unlike iterative RAG (no step-wise verification), KG-enhanced QA (single-pass context injection), and multi-agent verification (internal debate only), IteraTri uniquely enables atomic triplet verification and full step-wise traceability by iteratively decomposing questions, filtering evidence via reflection, and validating each reasoning step against a trusted knowledge graph.

3 Methodology

We propose the **IteraTri** (Iterative Triplet-Driven Reasoning) framework to detect hallucination in complex knowledge reasoning. As illustrated in Figure 1, IteraTri operates through three sequential stages: **Iterative Knowledge Extraction**, **Knowledge Reflection and Condensation**, and **Knowledge Verification and Factuality Judgment**. This structured approach ensures that the final answer is grounded in a verifiable chain of evidence.

3.1 Iterative Knowledge Extraction

This module decomposes a complex question into a series of verifiable sub-queries via a closed-loop Plan-Query-Extract process.

1. Query Planning (Thought Generation). Given a user question Q , the framework invokes a planner LLM, LLM_{plan} , to analyze the current reasoning state and determine the most critical piece of information required at next step. This generates a Thought that explicitly states the knowledge gap and the intent of the next query. The process at step t is formalized as:

$$\text{Thought}_t = LLM_{\text{plan}}(Q, \mathcal{H}_{t-1}),$$

where

$$\mathcal{H}_{t-1} = \bigcup_{i=1}^{t-1} \{(\text{Thought}_i, \text{Query}_i, \text{Knowledge}_i)\}$$

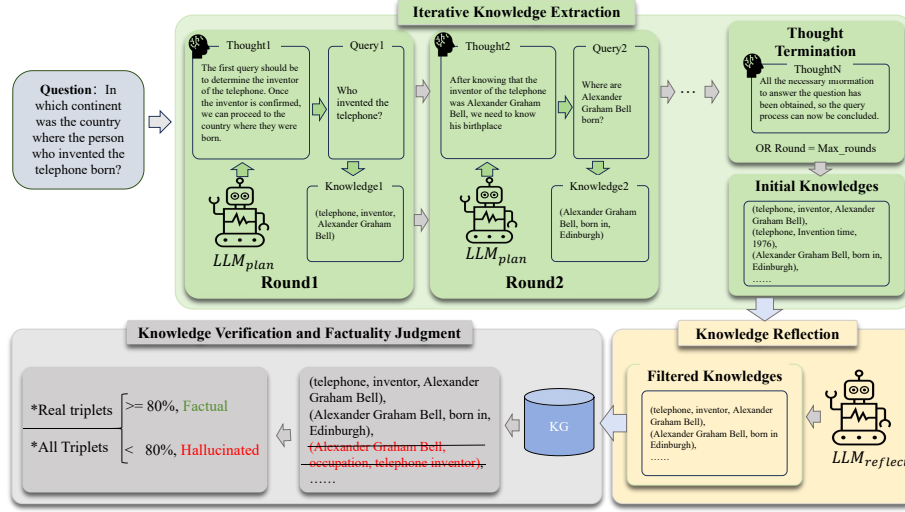


Fig. 1. The overall architecture of the IteraTri. The framework processes a user question through three core stages: (1) **Iterative Knowledge Extraction**, where an LLM planner decomposes the question across multiple rounds, generating structured knowledge triplets; (2) **Knowledge Reflection**, which condenses the extracted triplets into a coherent evidence chain; and (3) **Knowledge Verification**, where each triplet is checked against a trusted Knowledge Graph to produce a final, explainable factuality judgment.

is the history of all prior steps.

2. Targeted Query Formulation. Based on Thought_t , LLM_{plan} formulates a precise, retrievable natural language query:

$$\text{Query}_t = LLM_{\text{plan}}(\text{Thought}_t).$$

3. Structured Knowledge Extraction (Triplet Formation). Upon receiving the textual answer Answer_t for Query_t ,

$$\text{Answer}_t = LLM_{\text{plan}}(\text{Query}_t)$$

the framework does not adopt it directly. Instead, it prompts LLM_{plan} to distill the supporting facts into structured **knowledge triplets** $\mathcal{K}_t$. Each triplet is in the form $(\text{subject}, \text{relation}, \text{object})$, transforming unstructured text into verifiable atomic facts:

$$\mathcal{K}_t = \{(s, r, o)\} = LLM_{\text{plan}}(\text{Query}_t, \text{Answer}_t).$$

4. Iterative Loop. The triplets $\mathcal{K}_t$ are appended to the history $\mathcal{H}_t$, triggering the next cycle. This loop continues until LLM_{plan} determines that sufficient information has been gathered to answer Q or a pre-defined maximum number of rounds is reached. The output is an initial knowledge set tailored to the original

question:

$$\mathcal{K}_{\text{initial}} = \bigcup_{t=1}^T \mathcal{K}_t.$$

Prompt Design for Implementation. The iterative extraction process is governed by a structured prompt that is presented to LLM_{plan} . This prompt requires the model to output the four components (**Thought**, **Query**, **Answer**, **Knowledge**) in a strict cumulative format for each round t . The prompt operationalizes three key principles:

- **Single-focus querying:** Each query targets exactly one missing piece of information, **which prevents compound errors and simplifies subsequent verification of atomic facts.**
- **Cumulative triplet extraction:** All previously extracted triplets are carried forward in each **Knowledge** field.
- **Self-termination:** The model explicitly outputs "No further query needed" when sufficient information is gathered.

Prompt details: prompt follow a structured template with clear role definitions (planner agent), task instructions, and constraints on output format. The planner prompt enforces round-wise output of **Thought** $\rightarrow$ **Query** $\rightarrow$ **Answer** $\rightarrow$ **Triplets** and cumulative triplet tracking

3.2 Knowledge Reflection

The set $\mathcal{K}_{\text{initial}}$ may contain redundancies or facts that are only peripherally related to the final answer. To enhance the efficiency and precision of subsequent verification, we introduce a **Reflection** stage, executed by a dedicated reflection agent LLM_{reflect} .

This agent takes the original question Q and the initial knowledge set $\mathcal{K}_{\text{initial}}$ as input. Its task is to **select the minimal and logically coherent subset of triplets that are essential and sufficient to answer Q directly and concisely**. The process is formalized as follows:

$$\mathcal{K}_{\text{condensed}} = LLM_{\text{reflect}}(Q, \mathcal{K}_{\text{initial}}),$$

where $\mathcal{K}_{\text{condensed}} \subseteq \mathcal{K}_{\text{initial}}$.

Rationale. The reflection step not only reduces the computational cost of verification but also mimics the human "rethinking" process. By filtering out noise and focusing on the evidence chain, it significantly improves the robustness and accuracy of the final reasoning step. **The resulting $\mathcal{K}_{\text{condensed}}$ serves as input to the verification module (Section 3.3), ensuring that only relevant facts are checked against the KG.**

Prompt Design for Reflection. The reflection agent LLM_{reflect} operates based on a structured prompt that defines its core task: to filter the initial knowledge set $\mathcal{K}_{\text{initial}}$ by selecting only the triplets that are essential and sufficient for answering Q . The prompt provides explicit criteria for usefulness and mandates a

strict output format that includes a justification for each triplet’s categorization and final statistics. Key features include:

- **Binary usefulness classification:** Each triplet is judged as *Useful* or *Not Useful* based on its contribution to answering the question. **A triplet is considered useful if it directly participates in the reasoning chain or provides the necessary context without which the answer would be incomplete.**
- **Justification requirement:** For every triplet, a one-sentence reason must be provided, ensuring transparency.
- **Structured output:** The final output includes analysis, per-triplet categorization, statistics, and the condensed triplet list.

Prompt details: the reflection prompt follows a structured template with clear role definitions (reflection agent), task instructions, and constraints on output format. The reflection prompt enforces binary classification of triplets (Useful/Not Useful), requires one-sentence justification per triplet, and outputs a condensed, coherent evidence chain.

3.3 Knowledge Verification and Factuality Judgment

To ensure reliability, this module performs rigorous fact-checking on the condensed knowledge set $\mathcal{K}_{\text{condensed}}$ against a trusted Knowledge Graph (KG). The lexical gap between natural language phrases and KG identifiers is bridged by a neural-symbolic matching procedure that treats each candidate triple as an atomic textual unit and performs holistic similarity comparison.

1. KG Verification. For each natural language triplet $(s, r, o) \in \mathcal{K}_{\text{condensed}}$, we determine whether it corresponds to an existing triplet in the KG through the following multi-step process.

Embedding Model and Precomputation. All KG triples are pre-encoded into dense vectors to enable efficient similarity search. For each triple (e_h, p, e_o) in the KG, we concatenate the textual labels of its components into a single string (e.g., “ e_h p e_o ”) and encode this string using a pre-trained BERT model. All such embeddings are stored offline for fast cosine similarity computation during verification.

Head Entity Candidate Selection. For the subject phrase s :

- **Exact match check:** After applying case-insensitive normalization and trimming whitespace, we search for any KG entity whose label exactly equals s . If such an entity exists, it is selected as the **sole candidate** $\mathcal{C}_s = \{e^*\}$, and we skip fuzzy matching.
- **Fuzzy match:** If no exact match is found, we encode s using the same Sentence-BERT encoder to obtain its embedding $\mathbf{e}_s$. We then compute cosine similarity between $\mathbf{e}_s$ and all precomputed KG entity label embeddings. The **top-2 entities** with the highest similarity scores are selected as the set of candidates $\mathcal{C}_s$.

- **Early termination:** If $\mathcal{C}_s = \emptyset$ (which can occur when no exact match exists and the top similarity scores are extremely low), we immediately conclude that no matching triple exists and set $\text{Verify}(s, r, o) = 0$ without proceeding further.

Holistic Triple Matching. For each candidate head entity $e_h \in \mathcal{C}_s$, we examine all outgoing triples from that entity in the KG:

- Retrieve all outgoing triples $\mathcal{T}_{e_h} = \{(e_h, p, e_o)\}$ from the KG.
- For each $(e_h, p, e_o) \in \mathcal{T}_{e_h}$, we have its precomputed embedding $\mathbf{t}_{e_h, p, e_o}$.
- The input triplet (s, r, o) is serialized into the same format (e.g., “ $s \ r \ o$ ”) and encoded on-the-fly to obtain its embedding $\mathbf{q}$.
- Compute the cosine similarity between the query embedding and the KG triple embedding:

$$\text{sim} = \cos(\mathbf{q}, \mathbf{t}_{e_h, p, e_o}).$$

- If $\text{sim} \geq \theta$ (with $\theta = 0.96$), the candidate triple (e_h, p, e_o) is considered a **match** for (s, r, o) .

Verification Decision. The triplet (s, r, o) is assigned $\text{Verify}(s, r, o) = 1$ (i.e., verified as factual) if and only if both of the following conditions are satisfied:

1. **Head entity found:** $\mathcal{C}_s \neq \emptyset$ (at least one candidate head entity was identified).
2. **Matching triple exists:** There exists at least one candidate head entity $e_h \in \mathcal{C}_s$ such that, for some triple $(e_h, p, e_o) \in \mathcal{T}_{e_h}$, the cosine similarity satisfies $\text{sim} \geq \theta$.

If either condition fails, $\text{Verify}(s, r, o) = 0$, indicating that the triplet could not be verified against the KG.

Threshold Selection. The similarity threshold $\theta = 0.96$ was empirically determined on a development set. Values below 0.94 introduce a high rate of false positives (semantically similar but factually different triples), while values above 0.98 cause false negatives due to minor surface variations (e.g., “United States” vs. “USA”). $\theta = 0.96$ provides the best accuracy for the verification step itself and is fixed for all experiments.

2. Factuality Scoring and Judgment. An overall factuality support score ρ for the condensed set is computed:

$$\rho = \frac{\sum_{(s, r, o) \in \mathcal{K}_{\text{condensed}}} \text{Verify}(s, r, o)}{|\mathcal{K}_{\text{condensed}}|}.$$

Given a predefined threshold τ , a final judgment on the factuality of the prospective answer is made:

$$\text{Judgment} = \begin{cases} \mathbf{Factual}, & \text{if } \rho \geq \tau \\ \mathbf{Hallucinated}, & \text{otherwise.} \end{cases}$$

If the judgment is *Factual*, the framework synthesizes the final answer based on the verified triplets in $\mathcal{K}_{\text{condensed}}$. If *Hallucinated*, the framework either refrains from answering or flags the response as having insufficient factual support.

4 Experiments

We conducted comprehensive experiments to evaluate the effectiveness of the proposed IteraTri framework. Our experiments are designed to answer the following research questions:

- **RQ1:** Does IteraTri outperform all baseline methods in reducing hallucination and improving factuality in complex QA?
- **RQ2:** How does each component (e.g., iterative extraction, reflection) contribute to the overall performance?
- **RQ3:** How does IteraTri perform under different backbone LLMs and resource settings (zero-shot vs. few-shot)?

4.1 Experimental Setup

Datasets. We evaluate our method on three challenging benchmarks that require structured knowledge and complex reasoning, with a focus on verifiable factuality:

- **FactCHD** [14]: A benchmark designed for evaluating fact-conflicting hallucinations in knowledge-grounded QA. It provides questions with KG-grounded evidence chains. To ensure a rigorous and consistent evaluation of retrieval-based verification methods like ours, we focus exclusively on its fully verifiable **Wikidata subset**. We curated a set of 1,889 samples that are both retrievable against Wikidata and equipped with complete categorical metadata for per-type evaluation, making it an ideal testbed for assessing fine-grained reasoning and hallucination mitigation.
- **PathQuestion** [18]: A knowledge base question answering (KBQA) dataset specifically designed to evaluate multi-hop reasoning capabilities over knowledge graphs. To adapt it for our task of hallucination detection, we process it into a balanced binary classification benchmark. The factual half retains the original questions and correct answers. The hallucinated half is constructed by using prompt-guided LLM generation to produce answers that are semantically type-consistent but factually incorrect with respect to the knowledge base, which are then paired with the original questions. This transformation allows us to directly evaluate a model’s ability to distinguish fact from hallucination in a multi-hop reasoning context.
- **MetaQA** [19]: A large-scale knowledge base question answering dataset in the **movie domain**, containing over 400,000 questions that require multi-hop reasoning over a dedicated knowledge graph of movies, actors, and directors. Similar to our adaptation of PathQuestion, we process MetaQA into a balanced hallucination detection benchmark. The factual portion uses the original questions and answers. For the hallucinated portion, we employ a similar prompt-guided generation strategy with a large language model to create plausible but factually incorrect answers based on the movie-domain KG, ensuring semantic type alignment (e.g., generating a different movie title

or actor name). This enables a comprehensive evaluation of our framework’s ability to handle domain-specific, multi-hop queries and identify hallucinations within a complex, structured domain.

The choice of these datasets allows us to assess IteraTri’s capability across different reasoning complexities (e.g., multi-hop, comparison) and key tasks (i.e., accurate QA and explicit hallucination detection).

Baselines. We compare IteraTri against a comprehensive suite of competitive baselines. For a direct capability comparison, we evaluate the same suite of open-source LLMs (**GEMMA3-4B/12B** [20], **Llama3.1-8B** [21], and **Qwen3-8B** [22]) under both **Zero-shot** and **Few-shot** (with 4 in-context exemplars) CoT[23] prompting settings.

The few-shot prompt for baseline hallucination judgment follows a structured template with clear role definitions (judge agent), task instructions, and output format constraints. It provides four in-context QA-judgment examples, each demonstrating analysis steps and a binary factual/hallucinated verdict, guiding the model to produce a consistent judgment for the target question-answer pair.

We further compare against state-of-the-art specialized methods for hallucination detection and knowledge grounding:

- **TRUTH-TRIANGULATOR** [14]: Co-developed with FactCHD, this strong baseline employs a collaborative multi-agent architecture. It uses a *Truth-Guardian* for direct fact-checking and a *Truth-Seeker* for multi-hop evidential reasoning. A final arbitrator synthesizes their outputs for judgment.
- **SelfCheckGPT** [12]: This method detects hallucination via consistency checking. We generate five stochastic answers per question and use an NLI model to compute pairwise entailment, aggregating the scores into a final consistency signal.
- **InterrogateLM** [13]: This method first generates an answer, then uses the LLM to produce multiple follow-up questions about it. The consistency between the original answer and the LLM’s answers to these new questions is used to assess factuality.
- **RAG + LLM Judge**: This retrieval-based baseline first embeds the KG and retrieves the top-K triples most similar to the query. A powerful judge LLM (Qwen3-70B) is then prompted to determine hallucination by reasoning over this retrieved evidence subgraph.

All baseline methods are implemented following their original papers, using the best-performing backbone models and hyperparameters reported in each respective work.

Our Method (IteraTri) is instantiated using the same backbone LLMs as above, ensuring a fair comparison that highlights the gains from our framework’s architecture rather than base model capabilities.

Evaluation Metrics. The primary evaluation metric is the **Overall Accuracy** of factuality judgment. A claim (question-answer pair) is correctly judged if our framework’s binary verdict (**Factual** or **Hallucinated**) matches the ground-truth label.

Table 1. Performance comparison between IteraTri and baseline methods in accuracy. **Bold** and underlined numbers denote the best and second-best values, respectively.

Methods		FactCHD	PathQuestion	MetaQA
Zero-shot	Gemma3-4B	0.518	0.514	0.413
	Llama3.1-8B	0.471	0.552	0.526
	Qwen3-8B	0.641	0.609	0.115
	Gemma3-12B	0.526	0.549	0.451
ICL(4-shot)	Gemma3-4B	0.542	0.524	0.315
	Llama3.1-8B	0.498	0.562	0.532
	Qwen3-8B	0.608	0.612	0.436
	Gemma3-12B	0.641	0.655	0.393
RAG + LLM		0.652	0.680	0.593
InterrogateLLM		0.615	0.627	0.615
SelfCheckGPT		0.640	0.661	0.627
TRUTH-TRIANGULATOR		<u>0.735</u>	0.686	0.628
IteraTri	Gemma3-4B	0.712	0.785	0.753
	Llama3.1-8B	0.717	<u>0.790</u>	<u>0.786</u>
	Qwen3-8B	0.734	0.785	0.762
	Gemma3-12B	0.741	0.793	0.798

Implementation Details. We implement IteraTri using the official APIs or checkpoints of the backbone LLMs. For the knowledge graph verification stage, we use **Wikidata** as the trusted KG for FactCHD and PathQuestion, and the dedicated movie KG for MetaQA. The factuality judgment threshold τ is set to 0.8 based on a held-out validation set. The iterative extraction is limited to a maximum of 5 rounds. All prompts for Thought generation, Query formulation, Triplet extraction, and Reflection are carefully designed and kept consistent across models. Experiments are conducted on two NVIDIA A40 GPUs (each with 48GB of memory).

Evaluation Protocol. For each dataset and model, we report the performance of: (1) The base LLM under zero-shot and few-shot settings; (2) The specialized hallucination detection and grounding baselines (TRUTH-TRIANGULATOR, SelfCheckGPT, InterrogateLM, RAG+LLM Judge); (3) Our full IteraTri framework. This protocol allows for a clear isolation of the performance gain attributable to our framework.

4.2 Main Results

Table 1 presents the overall performance comparison on FactCHD, PathQuestion, and MetaQA datasets. Our proposed **IteraTri** framework consistently achieves superior accuracy across all evaluation settings.

Table 2. Ablation study on the IteraTri framework. ‘w/o reflection’ removes the knowledge reflection step; ‘w/o iteration’ performs single-round extraction.

Methods		FactCHD	PathQuestion	MetaQA
w/o reflection	Llama3.1-8B	0.694	0.776	0.768
	Gemma3-12B	0.723	0.781	0.776
w/o Iteration	Llama3.1-8B	0.672	0.732	0.736
	Gemma3-12B	0.634	0.754	0.746
IteraTri	Llama3.1-8B	0.717	0.790	0.786
	Gemma3-12B	0.741	0.793	0.798

Performance against base LLMs. IteraTri demonstrates substantial and consistent improvements over the zero-shot and few-shot (4-shot ICL) prompting of the same backbone LLMs. For instance, on FactCHD, IteraTri lifts the performance of Llama3.1-8B from 0.471 (zero-shot) to **0.717**, and improves Qwen3-8B from 0.641 to **0.734**. This underscores that the performance gain is attributed to our framework’s architecture rather than merely scaling the base model. **(This addresses RQ3 by showing IteraTri’s gains across different backbone models.)**

Performance against specialized baselines. IteraTri outperforms all strong, specialized baselines across all datasets. It surpasses the retrieval-based **RAG+LLM Judge**, the self-consistency method **SelfCheckGPT**, and the self-interrogation method **InterrogateLM** by a considerable margin. Compared to the multi-agent **TRUTH-TRIANGULATOR**, IteraTri achieves higher average accuracy on FactCHD (0.741 vs. 0.735 with Gemma3-12B) and substantially better results on both PathQuestion and MetaQA. Notably, with the Gemma3-12B backbone, IteraTri achieves the top scores of **0.741** on FactCHD, **0.793** on PathQuestion, and **0.798** on MetaQA, setting new state-of-the-art performance on these benchmarks. **(These results provide a strong affirmative answer to RQ1, demonstrating IteraTri’s effectiveness in hallucination detection.)**

Model-scale analysis. The results within IteraTri show a positive correlation between backbone model scale and final accuracy, with larger models (Gemma3-12B) consistently outperforming smaller ones (Gemma3-4B). This indicates that our framework effectively leverages enhanced base capabilities. **(This further elucidates RQ3, showing a consistent scaling trend.)**

4.3 Ablation Study

To dissect the contribution of each core component, we conduct an ablation study on two representative backbones, with results detailed in Table 2.

Impact of Knowledge Reflection (‘w/o reflection’). Removing the reflection module leads to a clear drop in performance across all datasets. For

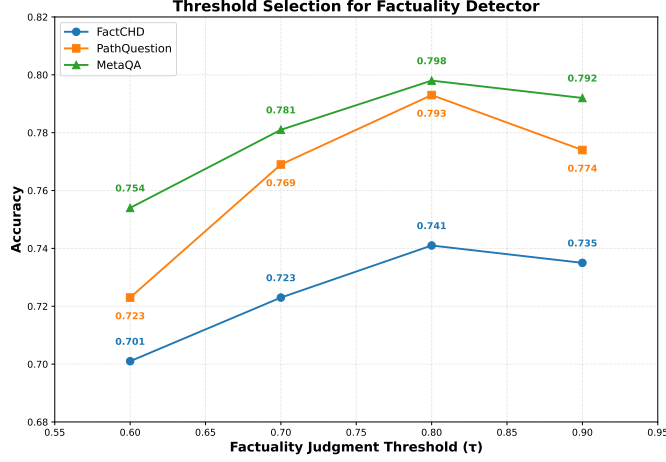


Fig. 2. Factuality Judgment Threshold Analysis.

example, with Llama3.1-8B, the FactCHD average score decreases from 0.717 to 0.694, and the PathQuestion score falls from 0.790 to 0.776. This confirms that the reflection step is crucial for condensing the extracted knowledge into a more precise and relevant evidence chain, thereby improving the efficiency and focus of the subsequent verification stage.

Impact of Iterative Extraction ('w/o iteration'). Disabling the multi-round iterative extraction causes the most significant performance degradation. The FactCHD average for Llama3.1-8B drops to 0.672, and for Gemma3-12B, it falls sharply to 0.634. This substantial decline validates our core hypothesis: complex questions cannot be reliably answered through a single query. The iterative process of planning, querying, and acquiring verifiable triplets is **essential** for decomposing and solving multi-hop reasoning problems. The full IteraTri framework, integrating both iteration and reflection, reliably recovers and exceeds the performance of the ablated variants, demonstrating the synergistic effect of its designed components.

(The ablation study directly answers RQ2, the contribution of both the iterative extraction and knowledge reflection components to IteraTri’s overall performance.)

The threshold τ directly controls the framework’s strictness in judging an answer as *Factual*. To determine its value, we conducted a cross-dataset sensitivity analysis on FactCHD, PathQuestion, and MetaQA, using the Gemma3-12B backbone. As shown in Figure 2, accuracy improves consistently across all three datasets as τ increases from 0.6 to 0.8, indicating that requiring a higher proportion of verified triplets leads to more reliable judgments. Beyond $\tau = 0.8$, however, accuracy plateaus or begins to decrease slightly on all datasets, suggesting that an overly strict threshold may discard correct answers supported by largely valid but partially incomplete evidence chains. This consistent trend across different datasets confirms that $\tau = 0.8$ provides a robust and generaliz-

able trade-off. Therefore, we select $\tau = 0.8$ as the optimal trade-off point for our experiments.

5 Limitations

While IteraTri achieves strong results on the evaluated benchmarks, several inherent limitations should be acknowledged.

Dependence on KG coverage. The verification module assumes that the target KG contains the necessary facts. For our datasets (FactCHD, PathQuestion, MetaQA), the associated KGs provide sufficient coverage; however, for domains with incomplete or rapidly evolving knowledge (e.g., newly released movies or recent events), false negatives may increase. This is an intrinsic limitation of any KG-grounded approach.

Computational cost. The iterative multi-round process introduces higher latency than single-pass methods. In our offline evaluation setting this overhead is acceptable, but it currently limits deployment in latency-sensitive applications. This trade-off is a direct consequence of prioritizing verification depth over speed.

Reflection sensitivity. The condensation step relies on a single LLM call to identify relevant triplets. While we observe that the reflection agent consistently produces coherent evidence chains (as evidenced by the ablation study), occasional over-filtering or under-filtering can occur. This sensitivity is inherent to the use of a zero-shot LLM for semantic filtering.

6 Conclusion

We present **IteraTri**, a framework for hallucination detection in knowledge-intensive QA by iteratively acquiring and verifying structured knowledge triplets. Its three-stage pipeline—iterative extraction, knowledge reflection, and KG-grounded verification—produces explainable factuality judgments with full traceability. Extensive experiments on multiple benchmarks demonstrate that IteraTri consistently outperforms strong baselines in detection accuracy, while providing transparent reasoning chains.

Future Work. To extend IteraTri’s applicability, in the future we can incorporate multi-source verification (e.g., textual corpora, live APIs) to reduce reliance on a single KG, explore efficient prompting strategies to lower latency, and investigate learning-based relevance scoring to further improve reflection robustness. We believe that triplet-driven, verifiable reasoning opens a promising path toward more reliable and auditable LLMs for knowledge-intensive tasks.

References

1. Wei Jason, Tay Yi, Bommasani Rishi, et al.: Emergent abilities of large language models. Transactions on Machine Learning Research (2022)

2. Joshua Maynez, Shashi Narayan, et al.: On faithfulness and factuality in abstractive summarization. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pages 1906–1919, Online. Association for Computational Linguistics (2020)
3. Marianna Martindale, Marine Carpuat, et al.: Identifying fluently inadequate output in neural and statistical machine translation. In Proceedings of Machine Translation Summit XVII: Research Track, pages 233–243, Dublin, Ireland (2019)
4. Ziwei Ji, Nayeon Lee, Rita Frieske, et al.: Survey of hallucination in natural language generation. *ACM computing surveys*, 55(12):1–38.653 (2023)
5. Xiang, Yuxuan, et al.: Reasoning path generation for answering multi-hop questions over knowledge graph. Asia-Pacific Web (APWeb) and Web-Age Information Management (WAIM) Joint International Conference on Web and Big Data, pages 195–209, Cham: Springer Nature Switzerland (2022)
6. Yuhao Zhang, Edward Gibson, and Forrest Davis: Can language models be tricked by language illusions? easier with syntax, harder with semantics. In Proceedings of the 27th Conference on Computational Natural Language Learning (CoNLL), pages 1–14, Singapore (2023)
7. Yue Zhang, Yafu Li, Leyang Cui, et al.: Siren’s song in the ai ocean: A survey on hallucination in large language models. *Computational Linguistics*, 51(1):1–45. (2025)
8. Vipula Rawte, Swagata Chakraborty, Agnibh Pathak, et al.: The troubling emergence of hallucination in large language models - an extensive definition, quantification, and prescriptive remediations. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pages 2541–2573, Singapore (2023)
9. Luyu Gao, Zhuyun Dai, Panupong Pasupat, et al.: Rarr: Researching and revising what language models say, using language models. In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics, volume 1, pages 16477–16508, Toronto, Canada (2023)
10. Yingjian Chen, Haoran Liu, Yinhong Liu, et al.: GraphCheck: Breaking long-term text barriers with extracted knowledge graph-powered fact-checking. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 14976–14995, Vienna, Austria (2025)
11. Niels Mündler, Jingxuan He, Slobodan Jenko and Martin Vechev: Self-contradictory hallucinations of large language models: Evaluation, detection and mitigation. In The Twelfth International Conference on Learning Representations (2024)
12. Potsawee Manakul, Adian Liusie, and Mark Gales: SelfcheckGPT: Zero-resource black-box hallucination detection for generative large language models. In The 2023 Conference on Empirical Methods in Natural Language Processing, Singapore (2023)
13. Yakir Yehuda, Itzik Malkiel, Oren Barkan, et al.: InterrogateLLM: Zero-resource hallucination detection in LLM-generated answers. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics, pages 9333–9347, Bangkok, Thailand (2024)
14. Xiang Chen, Duangzheng Song, Honghao Gui, et al.: Factchd: Benchmarking fact-conflicting hallucination detection. In IJCAI, pages 6216–6224, Jeju, South Korea (2024)
15. An, Ruize, et al.: Retrieval-Based Knowledge Consistency Validation for Fake News Detection. Asia-Pacific Web (APWeb) and Web-Age Information Manage-

- ment (WAIM) Joint International Conference on Web and Big Data, pages 436–451, Singapore: Springer Nature Singapore (2025)
16. Kelvin Guu, Kenton Lee, Zora Tung, et al.: Retrieval augmented language model pre-training. In Proceedings of the 37th International Conference on Machine Learning, pages 3929–3938, Virtual (2020)
 17. Nikhita Vedula and Srinivasan Parthasarathy: Face-keg: Fact checking explained using knowledge graphs. In Proceedings of the 14th ACM International Conference on Web Search and Data Mining, pages 526–534 (2021)
 18. Mantong Zhou, Minlie Huang, and Xiaoyan Zhu: An interpretable reasoning network for multi-relation question answering. In Proceedings of the 27th International Conference on Computational Linguistics, pages 2010–2022, Santa Fe, New Mexico, USA (2018)
 19. Haritz Puerto, Gzde Ahin, and Iryna Gurevych: Metaqa: Combining expert agents for multi-skill question answering. Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics, pages 3566–3580 (2023)
 20. Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, et al.: Gemma 3 technical report. Preprint, arXiv:2503.19786 (2025)
 21. Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al.: The llama 3 herd of models. Preprint, arXiv:2407.21783 (2024)
 22. An Yang, Anfeng Li, Baosong Yang, et al.: Qwen3 technical report. arXiv preprint (2025)
 23. Jason Wei, Xuezhi Wang, et al.: Chain-of-thought prompting elicits reasoning in large language models. In Advances in Neural Information Processing Systems, volume 35, pages 24824–24837. Curran Associates, Inc (2022)